

SE202L – Development Operations Lab Project

CareBase: AI-Powered Healthcare Appointment System

Deployment Document

GitHub Repository:

<https://github.com/Basileous/DevOps>

Submitted for SE202L – Development Operations Lab

Group Members

Name	Student ID
Basil Joseph	2024138
Muhammad Zesshan Nasim	2024495
Irtza Alyan	2024246
Mohit Kumar	2024306

Course Instructor

Mehmoona Amjad

1. Application Overview

CareBase is an AI-powered healthcare appointment management system built with Python Flask. It allows patients to browse available doctors, book appointments, and receive AI-driven symptom analysis with predicted conditions and precautions.

Who Would Use It

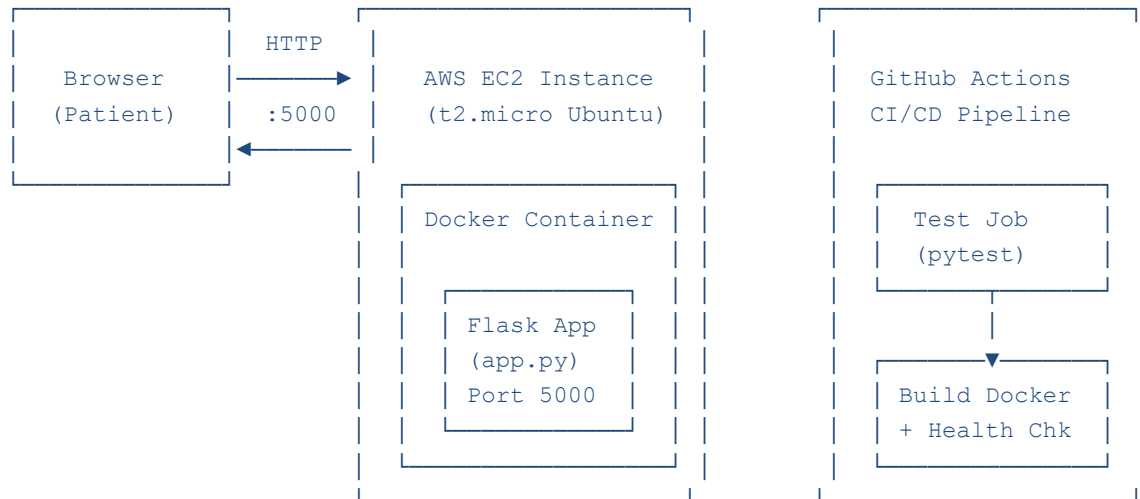
- Patients looking to book doctor appointments online
- Healthcare clinics managing their appointment schedules
- Anyone seeking quick AI-based symptom triage

API Endpoints

Method	URL	Description	Example Response
GET	/health	Health check	{"status": "ok"}
GET	/	Serves the frontend HTML page	HTML page
GET	/api/doctors	List all available doctors	{"doctors": [...]}
GET	/api/appointments	List all booked appointments	{"appointments": [...]}
POST	/api/appointments	Book a new appointment	{"message": "...", "appointment": {...}}
GET	/api/appointments/<id>	Get specific appointment	{"appointment": {...}}
DELETE	/api/appointments/<id>	Cancel an appointment	{"message": "Appointment cancelled"}
POST	/api/analyze	AI symptom analysis	{"specialty": "...", "predicted_conditions": [...]}

2. Architecture Diagram

The following ASCII diagram illustrates the overall architecture of the CareBase deployment:



3. Tools and Technologies

Tool	Purpose
Linux (Ubuntu 22.04)	Server operating system on AWS EC2 instance
Python 3.11	Programming language for the backend application
Flask	Lightweight web framework for building REST APIs
flask-cors	Enable Cross-Origin Resource Sharing for the API
pytest	Testing framework for automated unit tests
Git	Version control to track all code changes
GitHub	Remote repository hosting and collaboration
GitHub Actions	CI/CD pipeline for automated testing and Docker builds
Docker	Containerization to package the app with all dependencies
AWS EC2	Cloud hosting to make the app accessible from the internet
HTML/CSS/JS	Frontend technologies for the user interface

4. Local Setup Instructions

Follow these steps to clone the repository and run CareBase on your local machine.

Prerequisites

- Python 3.11 or higher installed
- Docker installed (for container testing)
- Git installed

Step 1: Clone the Repository

```
git clone https://github.com/Basileous/DevOps.git
cd DevOps
```

Step 2: Install Python Dependencies

```
pip install -r requirements.txt
```

Step 3: Run the Application Locally

```
python app.py
```

The app will start on <http://localhost:5000>. Open this URL in your browser.

Step 4: Run Tests

```
python -m pytest test_app.py -v
```

All 8 tests should pass.

Step 5: Run with Docker

```
docker build -t carebase:v1 .
docker run -d -p 5000:5000 carebase:v1
```

Visit <http://localhost:5000> to see the app running inside Docker.

5. CI/CD Pipeline Explanation

The CI/CD pipeline is defined in `.github/workflows/ci.yml` and triggers automatically on every push to the main branch.

Job 1: test

- Runs on ubuntu-latest
- Checks out the repository code
- Sets up Python 3.11
- Installs all dependencies from requirements.txt
- Runs `python -m pytest test_app.py -v` to execute all automated tests
- If any test fails, the pipeline stops and the build-docker job does NOT run

Job 2: build-docker

- Only runs if the test job passes (needs: test)
- Checks out the repository code
- Builds the Docker image: `docker build -t carebase:v1 .`
- Starts a container: `docker run -d -p 5000:5000 --name carebase-test carebase:v1`
- Waits 5 seconds for the container to start
- Runs a health check: `curl --fail http://localhost:5000/health`
- If the health check passes, the job succeeds (green checkmark)
- Stops and removes the test container

What Happens If a Test Fails?

If any pytest test fails, the test job shows a red X in the Actions tab. The build-docker job is skipped entirely because it depends on the test job. This prevents broken code from being built into a Docker image.

6. Deployment Steps

These are the exact steps taken to deploy CareBase on an AWS EC2 instance.

Step 1: Launch EC2 Instance

1. Log in to the AWS Management Console
2. Navigate to EC2 → Launch Instance
3. Configure the instance:
 - Name: CareBase-Server
 - AMI: Ubuntu Server 22.04 LTS
 - Instance type: t2.micro (Free Tier eligible)
 - Key pair: Create or select an existing key pair (e.g., carebase-key.pem)
 - Storage: 8 GB (default)

Step 2: Configure Security Group

Add these inbound rules:

- SSH: Port 22, Source: My IP
- Custom TCP: Port 5000, Source: 0.0.0.0/0 (allows internet access)

Step 3: Connect to the Instance

```
chmod 400 carebase-key.pem
ssh -i carebase-key.pem ubuntu@13.53.117.47
```

Step 4: Install Docker on the Instance

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ubuntu

# Log out and log back in for the group change to take effect:
exit
ssh -i carebase-key.pem ubuntu@13.53.117.47
```

Step 5: Clone the Repository

```
git clone https://github.com/Basileous/DevOps.git
cd DevOps
```

Step 6: Build the Docker Image

```
docker build -t carebase:v1 .
```

Step 7: Run the Container

```
docker run -d -p 5000:5000 --restart=always --name carebase carebase:v1
```

Step 8: Verify the Deployment

```
curl http://localhost:5000/health
```

Expected output: {"status":"ok"}

Step 9: Test from Your Local Machine

```
# From your local terminal (not the EC2 instance):
```

```
curl http://13.53.117.47:5000/health
```

```
# Open in browser:
```

```
http://13.53.117.47:5000
```

7. Testing Evidence

7.1 Pytest Results

All 8 pytest tests were run locally and passed successfully:

```
python -m pytest test_app.py -v
```

```
PS C:\Users\DELL\Desktop\Carebasefinal-main\Carebasefinal-main> python -m pytest test_app.py -v
===== test session starts =====
platform win32 -- Python 3.13.2, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\DELL\AppData\Local\Programs\Python\PS C:\Users\DELL\Desktop\Carebasefinal-main\Carebasefinal-main> python -m p
ytest test_app.py -v
===== test session starts =====
PS C:\Users\DELL\Desktop\Carebasefinal-main\Carebasefinal-main> python -m pytest test_app.py -v
PS C:\Users\DELL\Desktop\Carebasefinal-main\Carebasefinal-main> python -m pytest test_app.py -v
===== test session starts =====
PS C:\Users\DELL\Desktop\Carebasefinal-main\Carebasefinal-main> python -m pytest test_app.py -v
===== test session starts =====
platform win32 -- Python 3.13.2, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\DELL\AppData\Local\Programs\Python\PS C:\Users\DELL\Desktop\Carebasefinal-main\Carebasefinal-main> python -m p
ytest test_app.py -v
===== test session starts =====
platform win32 -- Python 3.13.2, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\DELL\AppData\Local\Programs\Python\PS C:\Users\DELL\Desktop\Carebasefinal-main\Carebasefinal-main> python -m p
ytest test_app.py -v
===== test session starts =====
PS C:\Users\DELL\Desktop\Carebasefinal-main\Carebasefinal-main> python -m pytest test_app.py -v
PS C:\Users\DELL\Desktop\Carebasefinal-main\Carebasefinal-main> python -m pytest test_app.py -v
PS C:\Users\DELL\Desktop\Carebasefinal-main\Carebasefinal-main> python -m pytest test_app.py -v
PS C:\Users\DELL\Desktop\Carebasefinal-main\Carebasefinal-main> python -m pytest test_app.py -v
===== test session starts =====
platform win32 -- Python 3.13.2, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\DELL\AppData\Local\Programs\Python\platform win32 -- Python 3.13.2, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\DEL
L\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\DELL\Desktop\Carebasefinal-main\Carebasefinal-main
collected 8 items
collected 8 items

test_app.py::test_health_endpoint PASSED [ 12%]
test_app.py::test_health_endpoint PASSED [ 12%]
test_app.py::test_get_doctors PASSED [ 25%]
test_app.py::test_create_appointment PASSED [ 37%]
test_app.py::test_double_booking PASSED [ 50%]
test_app.py::test_create_appointment_missing_fields PASSED [ 62%]
test_app.py::test_get_appointments PASSED [ 75%]
test_app.py::test_get_appointment_not_found PASSED [ 87%]
test_app.py::test_analyze_symptoms PASSED [100%]

===== 8 passed in 0.18s =====
PS C:\Users\DELL\Desktop\Carebasefinal-main\Carebasefinal-main> []
```

Figure 1 – All 8 pytest tests passing locally (Python 3.13, pytest 9.0.3)

7.2 GitHub Actions Pipeline

Both CI/CD jobs show green checkmarks in the GitHub Actions tab:

- test — All pytest tests passed
- build-docker — Image built and health check passed

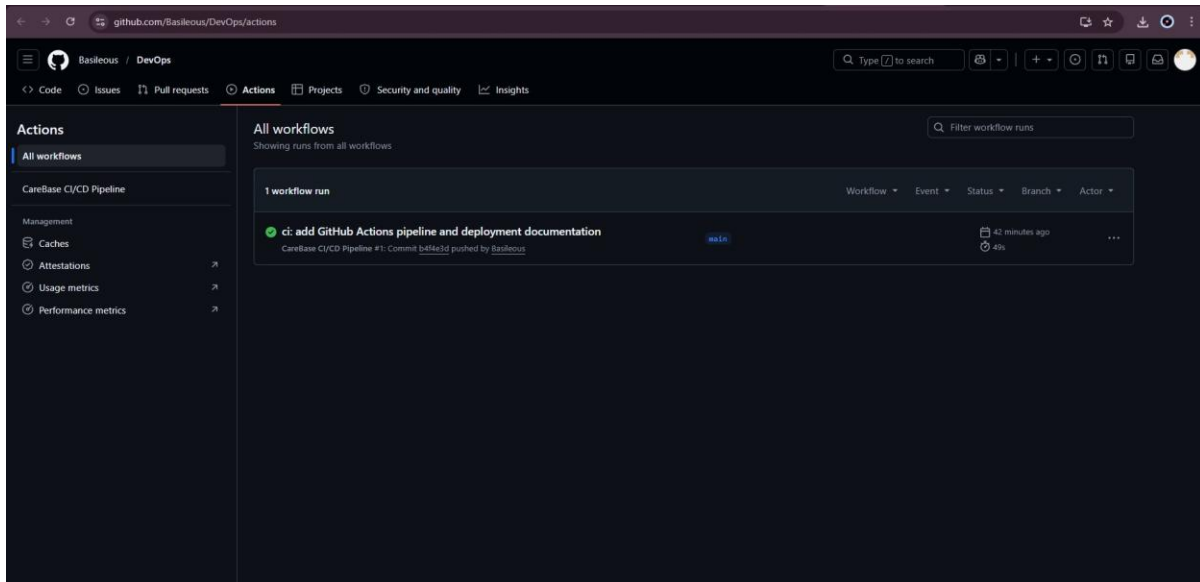


Figure 2 – GitHub Actions CI/CD Pipeline showing successful workflow run

7.3 Live Application

The application is live and accessible from the EC2 public IP address:

```
curl http://13.53.117.47:5000/health
# Output: {"status":"ok"}
```

```
curl http://13.53.117.47:5000/api/doctors
# Output: {"doctors": [...]}
```

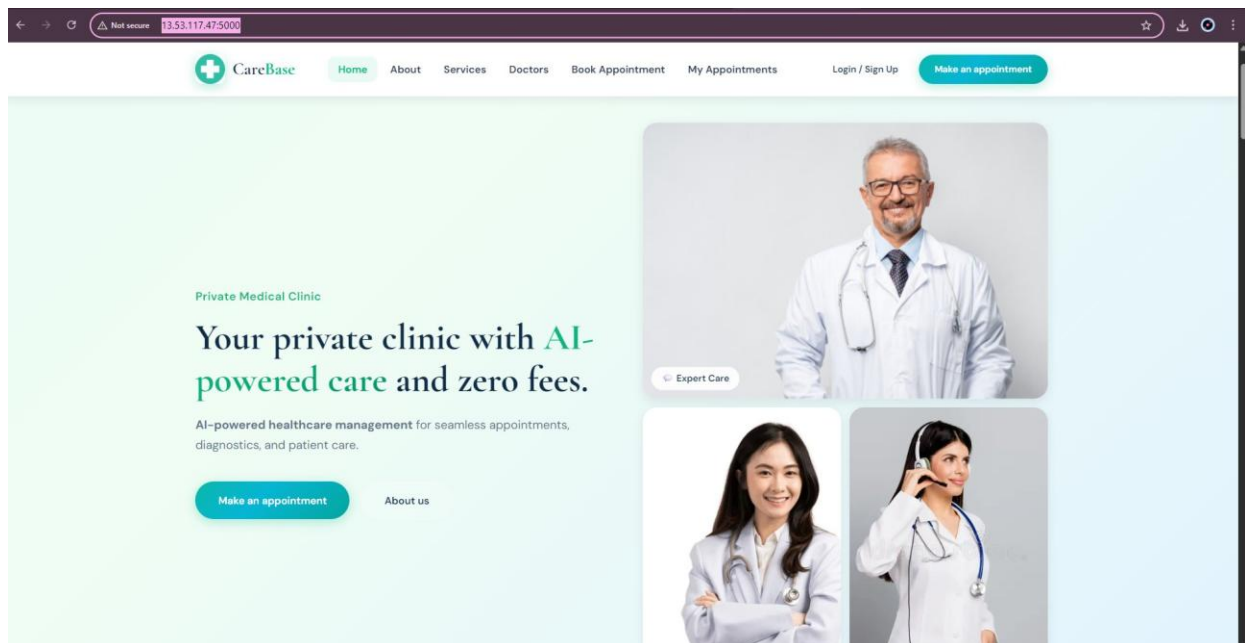


Figure 3 – CareBase live application running on EC2 at <http://13.53.117.47:5000>

8. Challenges and Solutions

Challenge 1: Docker Container Not Responding on Port 5000

Problem: After running `docker run -d -p 5000:5000 carebase:v1`, the app was not accessible from outside the EC2 instance.

Root Cause: The EC2 security group did not have an inbound rule for port 5000.

Solution: The AWS Console was used to navigate to EC2 → Security Groups → selected the instance's security group → Edit Inbound Rules → Added a Custom TCP rule for port 5000 with source 0.0.0.0/0. After saving, the app became accessible from the internet.

Challenge 2: GitHub Actions Build Failing Due to Missing Dependencies

Problem: The test job in GitHub Actions failed because flask-cors was not in requirements.txt initially.

Root Cause: flask-cors had been installed locally using `pip install flask-cors` but was not added to the requirements file.

Solution: The command `pip freeze > requirements.txt` was run to capture all dependencies, then cleaned up to include only the required packages. After pushing the updated requirements.txt, the pipeline passed successfully.

9. Lessons Learned

4. Docker makes deployment reproducible: By containerizing the app, it runs identically on a laptop and on the EC2 server. The Dockerfile captures every dependency and configuration step, eliminating the "it works on my machine" problem.
5. CI/CD prevents broken deployments: Having the GitHub Actions pipeline automatically run tests before building the Docker image prevented deploying broken code. When a bug was accidentally introduced, the pipeline caught it before it reached production.
6. Security groups are the cloud firewall: Understanding that EC2 security groups control network access was critical. Without opening port 5000, the app was running perfectly inside the container but was invisible to the outside world.
7. The `--restart=always` flag is essential for production: Without this flag, if the Docker container crashes or the EC2 instance reboots, the app stays down. With `--restart=always`, Docker automatically restarts the container, ensuring the app stays available.
8. Writing tests first makes development faster: By writing pytest tests early, each endpoint could be quickly verified without manually testing in the browser every time. This saved significant time during development.